

# ETNAir — Documentation générale

Plateforme de location de logements étudiants — projet ETNA C2W-CBI1

---

## Présentation du projet

ETNAir est une plateforme web inspirée d'Airbnb mais pensée spécifiquement pour les étudiants. Elle met en relation deux profils d'utilisateurs : d'un côté les **locataires**, qui peuvent rechercher des logements en filtrant par ville, prix ou dates de disponibilité, puis réserver directement en ligne ; de l'autre les **propriétaires**, qui publient leurs annonces, gèrent leurs photos, ajustent leurs prix et acceptent ou refusent les demandes de réservation.

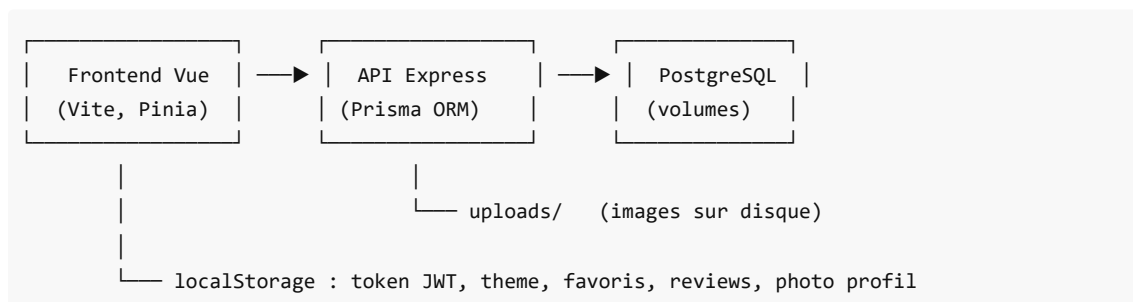
Au-delà de la mise en relation, la plateforme propose un système de favoris pour mettre de côté les annonces intéressantes, une section d'avis permettant aux locataires de partager leur expérience, ainsi qu'un tableau de bord personnel où chacun retrouve son activité. L'affichage des annonces peut se faire au choix sous forme de liste, de grille de cartes, ou directement sur une carte interactive utilisant Leaflet, ce qui permet de visualiser géographiquement l'offre disponible. Enfin, l'interface complète supporte un mode sombre et un mode clair, persistés dans le navigateur.

---

## Architecture technique

L'application repose sur une architecture trois tiers classique. Le **frontend** est une Single Page Application développée en Vue 3 avec Vite, qui consomme une API REST. Le **backend** est un serveur Express 5 sous Node.js 24, qui utilise Prisma comme ORM pour communiquer avec une base **PostgreSQL 16**. Toutes les images uploadées par les propriétaires sont stockées sur disque dans un volume persistant.

Côté navigateur, plusieurs informations sont conservées dans le `localStorage` : le token JWT d'authentification, la préférence de thème (sombre ou clair), la liste des annonces favorites, les avis publiés et la photo de profil de l'utilisateur en base64. Cette approche permet une expérience fluide et instantanée pour les fonctionnalités qui ne nécessitent pas systématiquement un aller-retour serveur.



Le développement local s'orchestre avec **Docker Compose**, qui démarre simultanément la base, l'API, le frontend et PgAdmin. Pour la production, le projet cible un cluster **Kubernetes** capable de gérer le scaling automatique, les backups, et le monitoring. L'intégration continue passe par **GitLab CI**, qui exécute la suite de tests Jest sur chaque push avant de déployer.

---

## Organisation du dépôt

Le code source est organisé en deux grandes parties principales. Le dossier `api/` contient toute la partie backend, depuis le serveur Express jusqu'aux schémas Prisma en passant par les migrations et le script de seed. Le dossier `frontend/` contient l'application Vue, avec ses vues, ses composants réutilisables, ses stores Pinia et son design system CSS centralisé.

À la racine, on trouve également le dossier `k8s/` qui regroupe tous les manifests Kubernetes (deployments, services, ingress, HPA, CronJob de backup), le dossier `k6/` avec les scripts de tests de charge, et enfin `docs/` qui contient la documentation complète du projet. Le fichier `docker-compose.yml` orchestre l'environnement de développement, tandis que `.gitlab-ci.yml` définit le pipeline CI/CD.

---

## Démarrage rapide

Pour lancer l'ensemble du projet en local, il suffit d'avoir Docker Desktop installé puis d'exécuter ces commandes :

```
git clone <repo>
cd ETNAIR
docker compose up -d
```

Au bout d'une trentaine de secondes, tous les services sont disponibles. Le **frontend** est accessible sur `http://localhost:5173`, l'**API** sur `http://localhost:3000`, et sa documentation **Swagger** sur `http://localhost:3000/api-docs`. Une interface d'administration **PgAdmin** est également disponible sur `http://localhost:5050` (login `harvey@etnair.xyz`, mot de passe `harvey`).

Au premier démarrage, un script de seed peuple automatiquement la base avec 35 utilisateurs fictifs et environ 127 annonces réparties dans 35 villes françaises, ce qui permet d'avoir immédiatement un dataset réaliste pour tester l'interface.

---

## Documentations détaillées

Chaque grand domaine du projet a sa propre documentation dans ce dossier `docs/`. Le fichier [frontend.md](#) couvre l'architecture Vue, les stores, les routes et le design system. Le fichier [backend.md](#) détaille tous les endpoints REST, le système d'authentification et la stratégie de tests. La documentation [database.md](#) explique le modèle de données Prisma, les migrations et le script de seed. Les aspects infrastructure sont couverts par [docker.md](#) pour l'environnement de développement et [kubernetes.md](#) pour la production. Enfin [tests.md](#) regroupe tout ce qui concerne la stratégie de test (Jest, Cypress, k6).

---

## Équipe

Le projet a été développé par quatre étudiants de l'ETNA. **Harvey M.** assure le rôle de co-fondateur et lead développeur, supervisant l'architecture technique globale. **Nelson P.** est co-fondateur et responsable produit, en charge de la vision fonctionnelle. **Alexandre C.** prend en charge le design UX/UI et veille à la cohérence visuelle de l'application. **Yamine I.** est l'ingénieur backend, spécialisé sur la base de données et la performance.

---

*Projet pédagogique ETNA — usage interne uniquement.*